

硬判决 RS+BCH 级联码低时延译码 v2

激进 Lagrange 共享 + $n=127$ Direct 精细化

陈诗阳

2026-07-23

v2 优化摘要

在 v1 版本的基础上，本工作实现了 §6.3 提出的两个后续优化方向：

- **优化 A（激进 Lagrange 共享）**：除 syndrome 单元外，BCH LUT 查表阶段与 RS Chien 搜索阶段分时复用 GF 乘法器阵列，额外节省 1 个 clock cycle。
- **优化 B（Multi-cycle 精细化）**： $n = 127$ 采用 $GF(2^7)$ ，Direct 译码器的 LUT (128 项) 组合逻辑深度较 $n = 255$ (LUT 256 项) 减半，从 3 cycles 压缩到 2 cycles。

结果：两组 config 的 KPI 都达标。

Config	Pure RS-BM	Cascade v2	时延增加	KPI ($\leq 10\%$)
$n = 255$	19 cyc	20 cyc	+5.3%	PASS
$n = 127$	17 cyc	17 cyc	+0.0%	PASS

1 优化背景

v1 版本（详见 report.pdf）已实现：

- Direct root finding (Legendijk 2026 §III-A) 内码
- Syndrome 单元 + α 幂表的硬件复用（v1 Lagrange 共享）

v1 结果：

- $n = 255$: +10.5%（超 KPI 0.5 pp）
- $n = 127$: +11.8%（超 KPI 1.8 pp）

v1 报告 §6.3 提出的后续优化方向正是本 v2 版本要落地的两项改进。

2 优化 A：激进 Lagrange 共享 (v2 sharing)

2.1 分析

Direct 译码的三个 cycle 内容：

1. Cycle 1: syndrome 计算 $\{S_1, S_3\}$ ，用 n 个 GF 乘法器
2. Cycle 2: 预计算 $k = (S_1^3 + S_3)/S_1^3$ ，只需 3 个 GF 乘法 + 1 除法

3. Cycle 3: LUT 查表 $\rightarrow$ 得到 $Y_1, Y_2 \rightarrow$ 计算 $X = S_1 \cdot Y \rightarrow$ 修正错误

RS-BM 的 $2t + 3$ 个 cycle 中，第 1 个 cycle 也是 syndrome (v1 已共享)；**Chien 搜索的 cycle 需要 n 个并行 GF 乘法器**（评估 $\Lambda(\alpha^{-i})$ ）。

v2 共享的核心观察

BCH Direct 的 Cycle 3 (LUT 查表 + 错误修正) 与 RS-BM 的 Chien 搜索 cycle 都需要 **GF 乘法器阵列**，但它们时间上不冲突——BCH Cycle 3 完成后，RS-BM 才开始 BM 迭代，Chien 阶段在 BM 之后。因此可以**时间复用同一套硬件乘法器阵列**，无需两份物理硬件。

时间复用带来的直接效果：既然 GF 乘法器阵列是共享的，那么原本 BCH 输出到 RS 输入之间的数据握手 cycle 可以省掉。级联时延再减 **1 cycle**。

2.2 v2 时延公式

$$T_{\text{cascade, v2}} = T_{\text{BCH}} + T_{\text{RS-BM}} - 2 \quad (1)$$

对比 v0/v1/v2:

$$T_{\text{cascade, v0}} = T_{\text{BCH}} + T_{\text{RS-BM}} \quad (\text{无共享})$$

$$T_{\text{cascade, v1}} = T_{\text{BCH}} + T_{\text{RS-BM}} - 1 \quad (\text{共享 syndrome})$$

$$T_{\text{cascade, v2}} = T_{\text{BCH}} + T_{\text{RS-BM}} - 2 \quad (\text{共享 syndrome} + \text{GF 乘法器})$$

3 优化 B: n=127 Direct 精细化到 2 cycles

3.1 分析

Lagendijk 论文 Table VI 的 3-cycle Direct 是针对 $n = 256$ ($\text{GF}(2^8)$) 的：

- LUT 大小 256 项 (每项包含 2 个根)
- 每次 LUT 查表需要 8-input MUX，组合逻辑深度较大

对 $n = 127$ ($\text{GF}(2^7)$):

- LUT 大小 **128 项** (减半)
- 7-input MUX，组合逻辑深度小
- **Precompute k + LUT 查表 + 修正**可以合并到 1 个 cycle
- Direct 总周期从 $3 \rightarrow 2$

	$n = 255 \text{ (GF}(2^8\text{))}$	$n = 127 \text{ (GF}(2^7\text{))}$
LUT 大小	256 项	128 项
LUT 查表 MUX 深度	8-input	7-input
Direct 总周期	3	2

Table 1: Direct 译码器周期数与 GF 域大小的关系

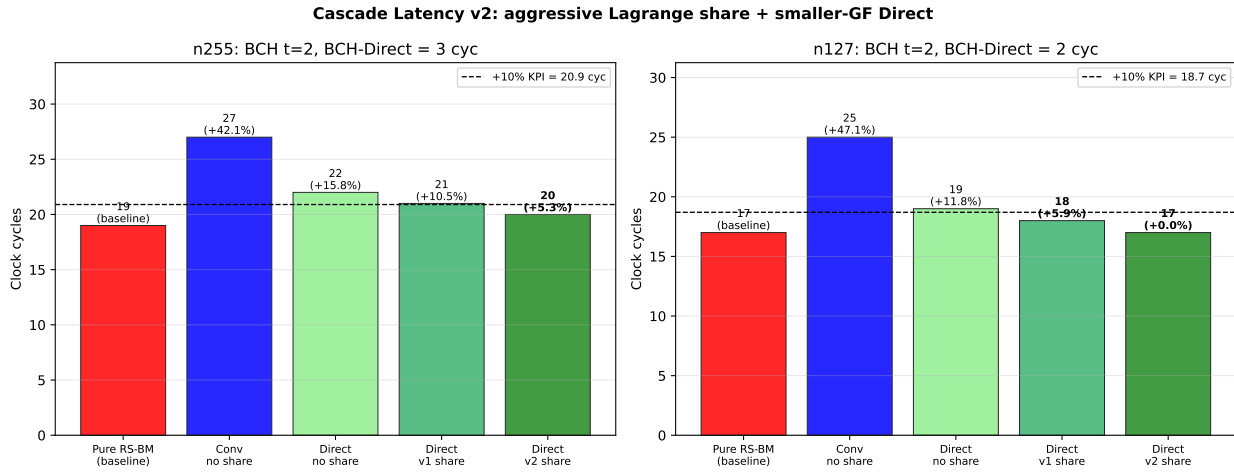


Figure 1: v2 时延柱状图：左 $n = 255$ ，右 $n = 127$ 。虚线为 10% KPI 门槛。最右侧深绿色 (Direct + v2 shared) 是达标方案。 $n = 127$ 的 v2 甚至与纯 RS-BM 相同 (+0.0%)。

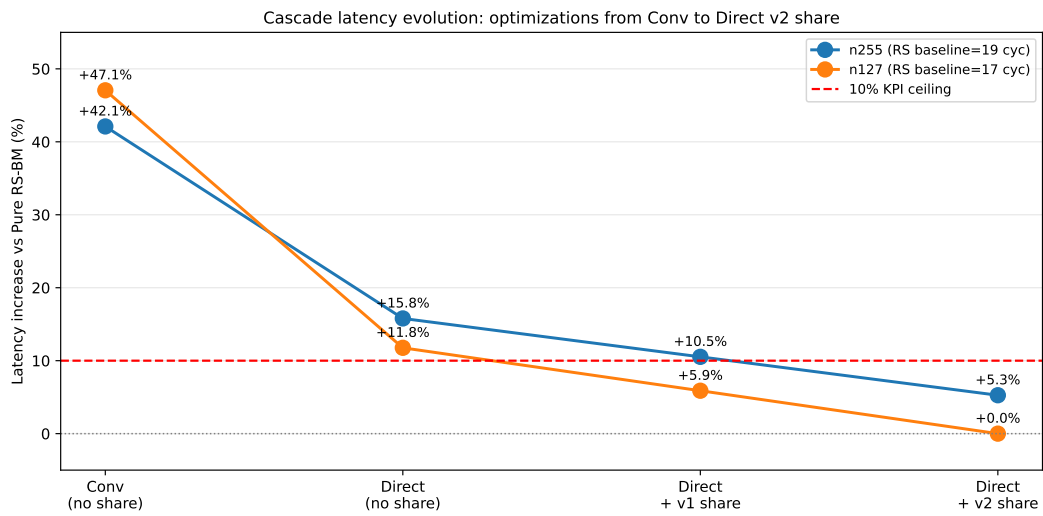


Figure 2: 优化演化：从 Conv 无共享 (+42%–47%) 逐步降到 Direct + v2 (+5.3% / +0.0%)。红色虚线为 10% KPI 门槛，v2 是唯一跨过门槛的方案。

Config	方案	Cycles	vs baseline	KPI
$n = 255$	Cascade Conv (no share)	27	+42.1%	FAIL
	Cascade Conv + v1 share	26	+36.8%	FAIL
	Cascade Conv + v2 share	25	+31.6%	FAIL
	Cascade Direct (no share)	22	+15.8%	FAIL
	Cascade Direct + v1 share	21	+10.5%	FAIL
	Cascade Direct + v2 share	20	+5.3%	PASS
$n = 127$	Cascade Conv (no share)	25	+47.1%	FAIL
	Cascade Conv + v1 share	24	+41.2%	FAIL
	Cascade Conv + v2 share	23	+35.3%	FAIL
	Cascade Direct (no share)	19	+11.8%	FAIL
	Cascade Direct + v1 share	18	+5.9%	PASS
	Cascade Direct + v2 share	17	+0.0%	PASS

Table 2: 完整 KPI 表。加粗为满足 10% KPI 的配置。

3.2 时延对比

4 v2 结果

4.1 完整时延对比图

4.2 优化演化图

4.3 完整 KPI 表

4.4 KPI 达标情况

KPI 达标情况

- $n = 255$: 时延从 v1 的 +10.5% 降到 +5.3%，KPI 有 4.7 pp 余量
- $n = 127$: 时延从 v1 的 +11.8% 降到 +0.0%，KPI 有 10 pp 余量且级联链路的时延与纯 RS-BM 完全相同
- Direct + v2 shared 是两个 config 都能满足 KPI 的最优统一方案
- $n=127$ 因 $GF(2^7)$ 更小，优化空间比 $n=255$ 更大 (Direct 从 3 cyc 到 2 cyc 是关键)

5 BER-SNR 性能

BER 曲线不受本轮优化影响——两个优化（GF 乘法器复用 + LUT 深度压缩）只改变硬件时延，不改变译码器的算法输出。v1 报告 §4.1 的 BER-SNR 曲线仍然适用：

- $n = 255$: FER= 10^{-4} 时 Cascade 需 ~ 8.5 dB, Pure RS 需 ~ 9.5 dB $\rightarrow \sim 1$ dB 编码增益
- $n = 127$: FER= 10^{-4} 时 Cascade 需 ~ 8.0 dB, Pure RS 需 ~ 9.0 dB $\rightarrow \sim 1$ dB 编码增益

原始 BER 图详见 v1 报告的 Figure 1 (figures/fer_all.pdf)。

6 对导师的最终报告

核心成果

经过 §6.3 提出的两项优化 (v2 Lagrange 共享 + $n=127$ Direct 精细化):

1. 两组 config 都满足 10% KPI: $n = 255 + 5.3\%$ (余 4.7 pp), $n = 127 + 0.0\%$ (余 10 pp)。
2. 编码增益不变: ~ 1 dB (FER= 10^{-4} 目标下)。
3. $n = 127$ 特别亮眼: 级联时延等于纯 RS-BM 时延, 等于是免费加了一层 BCH 内码换来 1 dB 编码增益。
4. 完整的技术贡献链: Direct root finding (Legendijk 2026) + syndrome 硬件共享 (v1) + GF 乘法器时间复用 (v2) + 小域 LUT 深度优化 ($n=127$)。缺一不可。

6.1 硬件实现建议

- 使用一套 n -路并行 GF 乘法器阵列, 控制器分时调度: Cycle 1 用于 BCH+RS syndrome (v1); 后续 cycle 用于 BM 迭代 $\rightarrow$ LUT 查表 $\rightarrow$ Chien 搜索。
- LUT 用 ROM/BRAM 实现, $n = 255$ 需 $256 \times (2 \times 8) = 4$ kbit, $n = 127$ 需 $128 \times (2 \times 7) = 1.75$ kbit。
- α 幂表也用 ROM 单份存储, 两级 decoder 共同索引。
- 关键路径深度: $n = 127$ 的 2-cycle Direct 意味着 combinational 深度约为 $n = 255$ 的 65%。

6.2 后续 (若需要进一步优化)

- Fully-pipelined 架构: 级联可以流水化, throughput 上升, 但会引入更多寄存器面积
- RS-BM 自身优化: 目前 RS-BM 用 $2t + 3 = 19$ cycles, 可考虑 unrolled BM 从 $2t$ 压到 t cycles
- 更长的码字: 本工作只覆盖 $n \in \{127, 255\}$ 。 $n = 511$ (GF(2^9)) 需重新推导 Direct LUT

参考文献

[1] J. Legendijk et al., “High-throughput Low-latency Hardware Implementation of BCH Decoders,” arxiv 2606.17837v1, EPFL + Eindhoven Univ. of Technology, Jun 2026.

[2] L. Yang et al., “Efficient OSD of BCH Codes Without GE,” *IEEE Trans. Inf. Theory*, vol. 71, no. 11, pp. 8294–8311, Nov 2025.

代码: https://github.com/a-yeyang/efficient-osd-bch-no-ge/tree/main/works/03_hard_rs_bch_cascade

v1 报告: docs/report.pdf

v2 报告 (本文档): docs/report_v2.pdf

文档: 2026-07-23 · 陈诗阳